

CISS358: Algorithm Analysis
Assignment 8

OBJECTIVES

1. Solve problems using greedy paradigm.
2. Prove correctness of greedy algorithm.

Create directory `a08/`. Put the latex files in `a08`. For the program for `a08q01`, put all the relevant files in `a08/a08q01/`

Q1. [GREEDY]

Implement Dijkstra's shortest path algorithm in C++. Use adjacency list to describe the graph. Assume the vertices are named $0, 1, 2, \dots, n-1$ where n is the number of nodes. You also need to keep track on the "distance from starting node to vertex i ", $d(i)$. You can use an array (you can use -1 for infinity or a number larger than then longest possible path). You also need to keep track on the weight from i to j , $w(i, j)$. You can either use a 2D array or a hashtable. Either one will achieve the runtime I mentioned in class (and in the notes), although if the graph is huge and sparse a hashtable is probably better. And for computing the solution, for each node n , you also need to keep track of "current best previous node of i to go back to the starting node", $\text{prev}(i)$. You can use an array for prev . Note that you will need to use a modified minheap, i.e., the minheap comes with a hashtable that keeps track on the values in the minheap.

Here's a sample execution:

```
4 5
0 1 10
0 2 11
1 2 200
1 3 100
2 3 12
0 3
0 2 3 23
```

The first line of input `4 5` is the number of nodes and number of edges. In this case there are 4 nodes. They are named $0, 1, 2, 3$. And there are 5 edges. The next 5 lines of output describes the weights in this graph. For instance `0 1 10` says that the edge from node 0 to node 1 has weight 10, `0 2 11` says that the edge from node 0 to node 2 has weight 11, etc. The graph is simple and undirected. So the weight going from node 0 to node 1 is the same as the weight going from node 1 to node 0. The last line of input `0 3` specifies the starting and ending node for the shortest path. The output `0 2 3 23` says that a shortest path is from 0 to 2 to 3 with path cost (i.e. sum of weights) of $11 + 12 = 23$.

Q2. [GREEDY]

You're working on a long research paper, and your draconian library will only allow you to have k books checked out at once. You know that you'll need more than this over the course of working on the paper, but at any point in time, you'd like to have ready access to the k books that are most relevant at that time. How should you decide which books to check out, and when should you return some in exchange for others, to minimize the number of times you have to exchange a book at the library?

As a concrete example, suppose $k = 2$ (i.e., you can only check out two books at a time. Suppose you need to refer to three books a, b, c . Furthermore, suppose you need to use these books in this order:

$$a, b, c, b, c, a, b$$

and suppose that you have books a on day 0 and b on day 1. Then on the day 2, since you need book c , you need to return either book a or book b in order to check out book c . Suppose you return book a . Therefore on day 2, you have books b and c . On day 3, you need book b , which you do have. On day 4, you need book c , which is already checked out. So on day 3 and 4, you do not need to return any books. But on day 5, you need book a . This means that you need to return either book b or c . Etc. The goal is (obviously) to prevent trips to the library.

Design an algorithm that minimizes library trips. Implement your algorithm in C++. Here's a sample execution:

```

2 3 7 0 1 2 1 2 0 1
1. 0->2
2.
3.
4. 2->0
5.
```

The first number, i.e., 2, in the input is the number of books you can check out. The second number, i.e., 3, is the number of books you need to use. The third number, i.e., 7, is the number of days you will be doing research. The rest, 0 1 2 1 2 0 1, is of (7) accesses to the books. Note that the books are numbered 0, 1, 2 if there are 3 books. The rest describes visits to the library. At the beginning, you already have books 0 and 1. So the output 1. 0->2 says that you return book 0 to check out 2. 2. says that you do not visit the library. This is followed by 3. which again says that you do not visit the library. 4. 2->1 says that you visit the library to return book 2 in order to check out book 1. Etc.

[ASIDE. This problem occurs in every situation where you need to work with more data than what a resource can handle at one time – i.e., there's a memory hierarchy. In the above library example, you can only hold two books at one time. The library can hold more. In the case of computer architecture, main memory and hard drive. It's better to have

data in main memory because that's faster than hard drive. But the problem is that your computer has a lot less capacity in main memory than the hard drive. For modern computer architecture, there's on-chip cache memory which is faster than main memory. Likewise, a database system stores pages of data in main memory to create a buffer pool and attempts to predict what pages of data stored in the hard drive in order to prevent replacing buffer pool pages with new pages from the hard drive. In the above library example, when you return a book, say a , already checked out for a new book, say c , we say that your algorithm is **evicting** a . Sometimes a is also called a **victim**. The overall algorithm that exchanging a piece of data in cache with another is frequently called a **replacement algorithm**. Note that for our library problem, it assumes that you know ahead of time what book will be needed in the future. In real life, frequently only an approximate access pattern is known, i.e., the access pattern is given in terms of probabilistic measures.]

Q3. [GREEDY]

You are starting a consulting company that needs to obtain licenses for n different pieces of software. Due to regulations, you can only obtain these licenses at the rate of at most one per month.

Each license is currently selling for a price of \$100. However, they become more expensive according to exponential growth curves: in particular, the cost of license i increases by a factor of $r_i > 1$ each month, where r_i is a given parameter. This means that if license i is purchased t months from now, it will cost $100 \cdot r_i^t$. We will assume that all the price growth rates are distinct; that is, $r_i \neq r_j$ for licenses $i \neq j$ (even though they start at the same price of \$100).

Given that you can only buy at most one license a month, in which order should you buy the licenses so that the total amount of money you spends is as small as possible?

Describe an algorithm that takes the n rates of price growth $r_0, r_1, \dots, r_{n-1}$, and computes an order in which to buy the licenses so that the total amount of money spent is minimized. State the runtime of your algorithm. The running time of your algorithm should be polynomial in n .

For this problem, you only need to describe your algorithm. Make sure you describe your algorithm clearly.

(Typeset your answer in `q03.tex`.

SOLUTION.

Q4. Write a complete proof that Prim's MST algorithm is correct. (This was given in class. Make sure you write up the proof and check it thoroughly, filling in gaps when necessary so that proof is complete.)

(Typeset your answer in `q04.tex`.

SOLUTION.